



FOCUSED AUDIT · SAMPLE REPORT

Does the 1 kHz servo loop hold?

Meridane Robotics · MR-7 collaborative arm · JC-3 joint controller, firmware 3.4.1

COMMISSIONED BY

Lead investor, Series B — on
behalf of the investment
committee

ENGAGEMENT

DYR-FA-2026-014

DELIVERED

September 2026

CLASSIFICATION

Confidential · mutual NDA

THE VERDICT

The servo loop does not hold its 500 μs bound as built: its worst-case response is **886 μs** . It holds, with 12 μs to spare, once one critical section is removed. The 2 ms safe stop cannot be met by the current design at any priority — and can be met by a different one.

CRITICAL

The servo loop misses its bound

A 180 μs critical section in telemetry, on the CAN transmit lock the servo shares, blocks it. Worst case 886 μs against a stated 500. The bench never saw it; the analysis shows why.

Finding 1 · page 7

CRITICAL

The safe stop cannot meet 2 ms

Faults are polled every 2,000 μs , so detection alone can consume the whole budget. Worst case fault to stop: 2,970 μs . No priority change fixes a polling period.

Finding 2 · page 9

MAJOR

The margin rests on measured WCETs

After the fixes the servo has 12 μs of margin: its WCET may grow from 210 to 222 μs and no further. The 210 is a bench maximum, not a bound.

Finding 3 · page 10

886

μs servo worst case
stated: 500

2,970

μs fault to stop
required: 2,000

18

μs blocking budget
as built: 180

488

μs servo after two changes
deadline: 500

What it means for the investment

The datasheet's latency claim and the safety case's stop time both rest on figures the firmware does not deliver as built. Both are fixable in software, with two contained changes and no new hardware. We would make those changes, and a WCET re-measurement with a stated margin, conditions of the next milestone.

Where this verdict stops

It holds under the reconstructed model, with execution times taken from the vendor's bench traces. It does not certify those execution times, the hardware stop path, or the arm's stopping distance. Page 12 sets out the boundary in full.

One question, investigated to the bottom.

Does the MR-7's joint controller deliver the timing its datasheet and its safety case state — under worst-case conditions, not observed ones?

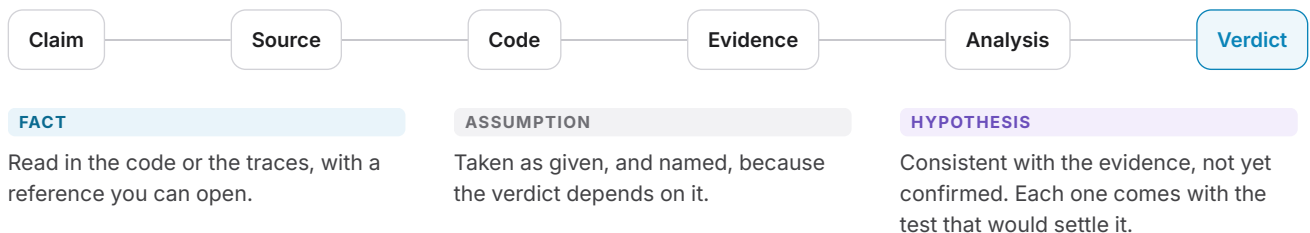
Examined

- ✓ Firmware 3.4.1, at the commit tagged for the Series B data room
- ✓ The RTIC application: every task, priority, shared resource and ceiling
- ✓ The CAN driver and the telemetry path that batches over it
- ✓ A 72-hour bench trace: 259 million servo cycles, DWT cycle counts per task
- ✓ The datasheet (v2), the safety concept, and the investor deck

Not examined

- Instrumentation on our own bench — the vendor's traces were used as given
- The hardware stop path: comparator, gate driver, brake
- The CAN bus as a network: arbitration, error frames, other nodes
- Code outside the control and safety paths

How every claim is handled



Four statements, and what stands behind each.

CLAIM, AS MADE	WHERE	WHAT THE EVIDENCE SUPPORTS	VERDICT
"A deterministic 1 kHz servo loop"	Datasheet v2, p. 4	The loop is released at 1 kHz by TIM2 FACT . Its completion is not deterministic: it varies with the phasing of telemetry against the loop.	Partly
"Worst-case control latency below 500 μ s"	Datasheet v2, p. 4	Worst-case response 886 μ s under the reconstructed model. The 431 μ s quoted alongside it is the largest value observed on the bench.	Does not hold
"Safe stop within 2 ms of a fault"	Safety concept §5.2 · deck, slide 11	Faults are polled at 500 Hz. Worst-case fault to stop command: 2,970 μ s; 2,624 μ s even with no blocking at all.	Does not hold
"No dynamic memory on the control path"	Datasheet v2, p. 6	The crate is <code>no_std</code> with no allocator; no task body on the control or safety path allocates FACT .	Holds

A claim that holds is reported with the same care as one that fails. A verdict that only finds faults is not independent.

Five tasks, one shared lock.

Rebuilt from the RTIC application, not from the architecture diagram. Priority falls from top to bottom. All times in microseconds.

TASK	ROLE	C	T	D	J	B	SOURCE
current	Current-control ISR, 20 kHz	18	50	50	0	0	trace dwt-max.csv · ISR TIM1_UP
can_rx	Fieldbus receive, sporadic $\geq 250\text{ }\mu\text{s}$	40	250	250	0	0	trace dwt-max.csv · FDCAN1_IT0
servo	Position and velocity loop, 1 kHz	210	1,000	500	12	180	app.rs:212 · trace · TIM2 jitter
safety	Safe-stop monitor, polled at 500 Hz	60	2,000	2,000	0	180	app.rs:261 · trace
telemetry	Telemetry batch over CAN, 100 Hz	300	10,000	10,000	0	0	telemetry.rs:88 · trace

The lock that matters

`servo`, `safety` and `telemetry` all declare the shared resource `can_tx` **FACT**. RTIC gives it the ceiling of its highest user, the servo. When telemetry holds it — for 180 μs while it transmits a batch — neither the servo nor the safety monitor can run, though both outrank telemetry.

Where the numbers come from

C is the largest DWT cycle count per task over the 72-hour trace **ASSUMPTION**. **J** is the release jitter of TIM2, measured at 12 μs . **B** is the longest critical section on `can_tx` by a lower-priority task: telemetry's batch, 180 μs .

Worst-case response, computed — not observed.

Fixed-priority response-time analysis with release jitter and blocking: Joseph and Pandya (1986), Tindell, Burns and Wellings (1994). Computed with DY-WCET, in integer arithmetic, and checked job by job against an independent simulation of the schedule. The two agree on every figure in this report.

The vendor's model — no blocking

TASK	D	R	
servo	500	482	meets
safety	2,000	624	meets

The firmware as built

TASK	D	R	
current	50	18	meets
can_rx	250	76	meets
servo	500	886	misses
safety	2,000	970	meets
telemetry	10,000	1,672	meets

The servo loop, every iteration

$w^0 = C + B = 210 + 180 = 390$. Each step adds the interference the two higher-priority tasks can impose within the current window. The analysis stops when a step returns the value it was given.

```

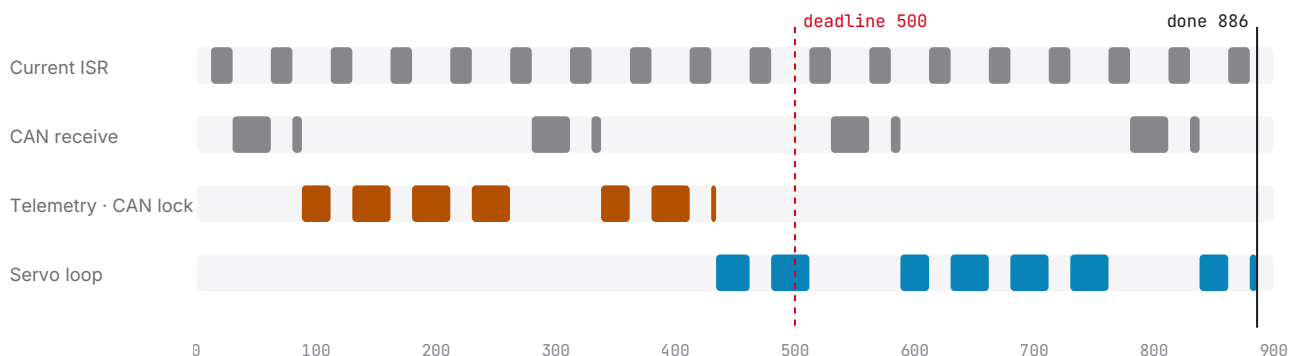
w = 390  [390/50]·18 = 8·18 = 144  [390/250]·40 = 2·40 = 80  → 614
w = 614  [614/50]·18 = 13·18 = 234  [614/250]·40 = 3·40 = 120 → 744
w = 744  [744/50]·18 = 15·18 = 270  [744/250]·40 = 3·40 = 120 → 780
w = 780  [780/50]·18 = 16·18 = 288  [780/250]·40 = 4·40 = 160 → 838
w = 838  [838/50]·18 = 17·18 = 306  [838/250]·40 = 4·40 = 160 → 856
w = 856  [856/50]·18 = 18·18 = 324  [856/250]·40 = 4·40 = 160 → 874
w = 874  [874/50]·18 = 18·18 = 324  [874/250]·40 = 4·40 = 160 → 874

```

fixed point: $w = 874$ $R = w + J = 874 + 12 = 886 \mu s$ deadline 500 μs missed by 386 μs

The servo loop misses its bound by 386 μ s.

The worst case needs three things at once: telemetry has just taken the CAN lock, and the current loop and a CAN frame arrive with the servo's release. Nothing in the firmware prevents that alignment.



The servo's worst case as built, on the arrival axis. Its release comes 12 μ s late; telemetry's critical section runs first, and the current loop and CAN receive preempt everything. The servo finishes at 886 μ s.

Evidence

- `app.rs:212` declares `shared = [can_tx]` on the servo task **FACT**
- `telemetry.rs:88` holds `can_tx` for the whole batch transmit **FACT**
- The batch takes 180 μ s at the trace's maximum **ASSUMPTION**

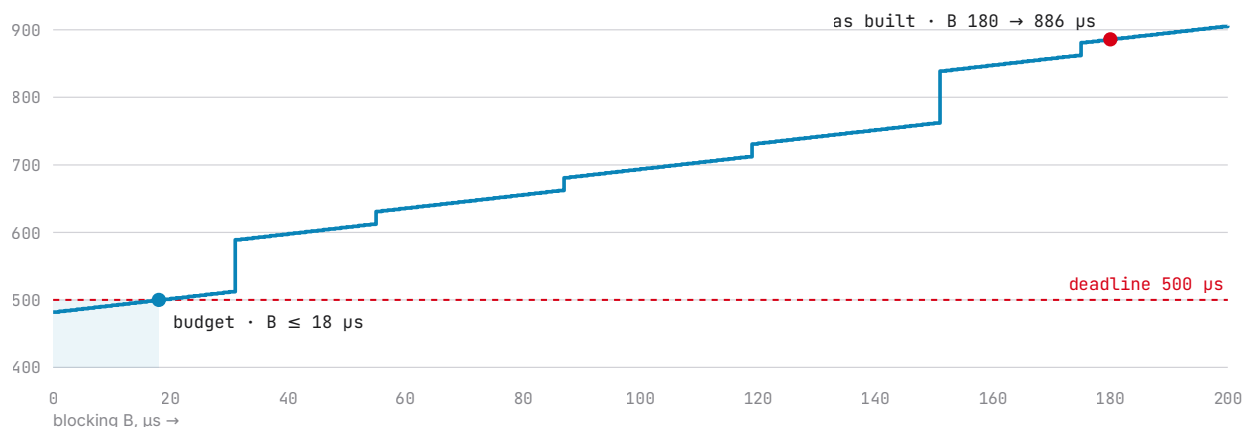
What would confirm it — and what would rule it out

Confirm: drive a GPIO high at servo release and low at completion, and trigger telemetry's batch 1 μ s before a servo release. The scope should read about 886 μ s.

Rule out: a servo path that never reaches `can_tx` under load. The code shows that it does.

Why 259 million cycles never showed it.

The bench was not wrong. It measured one phasing. Telemetry and the servo share a timer base, so on the bench telemetry's batch always begins 3.2 ms into the servo's period — never just before a release. A change to the telemetry rate, a reset sequence, or clock drift between two boards changes that phasing; the analysis covers every phasing at once.



Worst-case response of the servo loop against the blocking it suffers. The response climbs in steps — each step one more interference the window admits — and crosses the deadline at a blocking of 19 μs. As built, the blocking is 180 μs.

The budget. The servo meets 500 μs if, and only if, no lower-priority task holds a lock it needs for more than **18 μs**. That number is the design rule this firmware needs, and it is ten times smaller than what telemetry does today.

The safe stop cannot meet 2 ms — at any priority.

The safety monitor reads the fault input every 2,000 μs . A fault that arrives just after a read waits a full period before it is seen, then waits for the monitor's own worst-case response. The polling period alone equals the requirement.



Fault to stop command, worst case. As built: 2,000 μs to the next poll, then 970 μs of response — 2,970 μs . With a fault line on an interrupt at the highest priority, the software path is 6 μs .

The arithmetic

Worst case = $T + R = 2,000 + 970 = 2,970 \mu\text{s}$.
 Removing the blocking would bring R down to 624 μs and the total to 2,624 μs : still over. Raising the monitor's priority shortens R, not T.

What would confirm it — and what would rule it out

Confirm: inject the fault 10 μs after the monitor reads its input, and time fault to gate-driver disable on a scope. Expect about 2,970 μs .

Rule out: a hardware path that stops the arm without the monitor. The schematic shows none **HYPOTHESIS** — to be confirmed with the electronics lead.

After the fixes, the margin is 12 μ s. It rests on a measurement.

Every response time in this report is exact for the execution times it is given. Those execution times are the largest values the bench recorded, not proven upper bounds. With the two fixes in place, the servo loop meets 500 μ s as long as its execution time stays at or below 222 μ s — 12 μ s above the 210 measured.

Why the 210 may be optimistic

- The trace ran with the D-cache warm and flash wait states at their nominal setting **FACT**
- The encoder-error path was not exercised in the 72 hours **FACT**
- On a Cortex-M7, a cold-cache pass through the control law can cost tens of microseconds more

HYPOTHESIS

What we would require

A WCET for the servo task with a stated method and a stated margin: either a measurement campaign that forces the encoder-error path and a cold cache, or a static analysis of the compiled binary. Either way, the result must clear 222 μ s with room to spare.

Why this matters to a buyer. A margin is only as good as the number it is measured against. Twelve microseconds is enough, if the 210 is right. The report says so plainly rather than rounding it into confidence.

Two changes. Then every deadline holds.

1 Take telemetry off the shared lock

Replace the `can_tx` lock with a lock-free single-producer queue per sender, drained by one transmit handler. The servo and the safety monitor are then never blocked by telemetry: B falls from 180 μ s to 0.

Confirms it: the GPIO test of Finding 1 reads under 500 μ s at every phasing. Rules it out: any remaining `shared = [can_tx]` on a control-path task.

2 Stop on an interrupt, not a poll

Route the fault comparator to an EXTI line at the highest priority, and let its handler disable the gate driver directly — before it tells anything else. The software path from fault to disable becomes 6 μ s.

Confirms it: fault to gate-disable under 50 μ s on a scope. Rules it out: a gate driver that needs the CAN bus to turn off.

3 Bound the servo's execution time

Re-measure with a cold cache and the encoder-error path forced, or analyse the binary statically. Clear 222 μ s with a stated margin.

The system after changes 1 and 2

TASK	D	R	
fault_exti	100	6	meets
current	50	24	meets
can_rx	250	82	meets
servo	500	488	meets
safety	2,000	630	meets
telemetry	10,000	1,678	meets

The fault handler takes 6 μ s from every task below it; the table includes that cost. Every figure is recomputed with the new task in place — a fix that breaks another deadline is not a fix.

What this report establishes — and what it does not.

Established

- ✓ Under the reconstructed model, the servo loop's worst-case response is 886 μ s: the 500 μ s claim does not hold as built
- ✓ The polled safe stop cannot meet 2 ms under any priority assignment
- ✓ With changes 1 and 2, every task meets its deadline, by the margins in the table on page 11
- ✓ The servo's blocking budget is 18 μ s, and its execution-time budget after the fixes is 222 μ s

Not established

- The execution times themselves: they are bench maxima, taken as given
- The hardware stop path: comparator, gate driver and brake latencies
- CAN network timing between nodes
- The arm's stopping distance, and any claim of conformity to a safety standard
- Code outside the control and safety paths

How to read it. Every verdict in this report is exact for its stated inputs and silent beyond them. Where an input is an assumption, the report names it, and says what would replace it with a fact.

The core figures, in nine lines.

The analysis behind this report runs in DY-WCET, open source under Apache-2.0 or MIT. Priority follows the order of `push`. Every other figure in this report comes from the same set: set the blocking to 0 for the vendor's model, or push the fault handler first for the system after the changes.

```
use dy_wcet::{Task, TaskSet};

let mut mr7 = TaskSet::new();
mr7.push(Task::new(18, 50).named("current"))?;
mr7.push(Task::new(40, 250).named("can_rx"))?;
mr7.push(Task::new(210, 1000).deadline(500).jitter(12).blocking(180).named("servo"))?;
mr7.push(Task::new(60, 2000).blocking(180).named("safety"))?;
mr7.push(Task::new(300, 10000).named("telemetry"))?;

mr7.response_of(2); // Refused(ExceedsDeadline(886))
mr7.response_of(3); // Bounded(970)
```

Appendix B · Evidence index

firmware/src/app.rs:212	servo task: priority 3, shared = [can_tx]
firmware/src/app.rs:261	safety task: polled at 500 Hz, shared = [can_tx]
firmware/src/telemetry.rs:88	<code>can_tx.lock(tx tx.transmit_batch(..))</code> — the 180 μ s section
traces/bench-72h/dwt-max.csv	per-task DWT maxima over 259 million servo cycles
docs/MR-7-datasheet-v2.pdf, p. 4	the 1 kHz and 500 μ s claims
safety/concept-v1.3.pdf §5.2	the 2 ms safe-stop requirement

Know what is real.

Independent technical due diligence for investors, founders and engineering teams. Every claim a technology makes, traced to its code, its tests and its evidence — and a written verdict on what holds.

Snapshot

\$5,000 · 5 business days

What does this technology actually do, and what does its evidence support?

Focused Audit

\$12,000 · 2–3 weeks

Does one critical property — timing, determinism, concurrency — actually hold? This report is one.

Due Diligence

\$25,000 · 3–4 weeks

Is the technology what the company says it is, and what could break the investment?

dyresearch.github.io · connect@axonos.org · linkedin.com/in/dyresearch

What you show us stays yours.

A mutual NDA is signed before the first file, on every engagement. Access is read-only and ends at delivery; nothing is kept but the report, and deletion is confirmed in writing. The report is yours, and nothing — the findings, your name, or the fact that you engaged us — is ever published without your written consent.